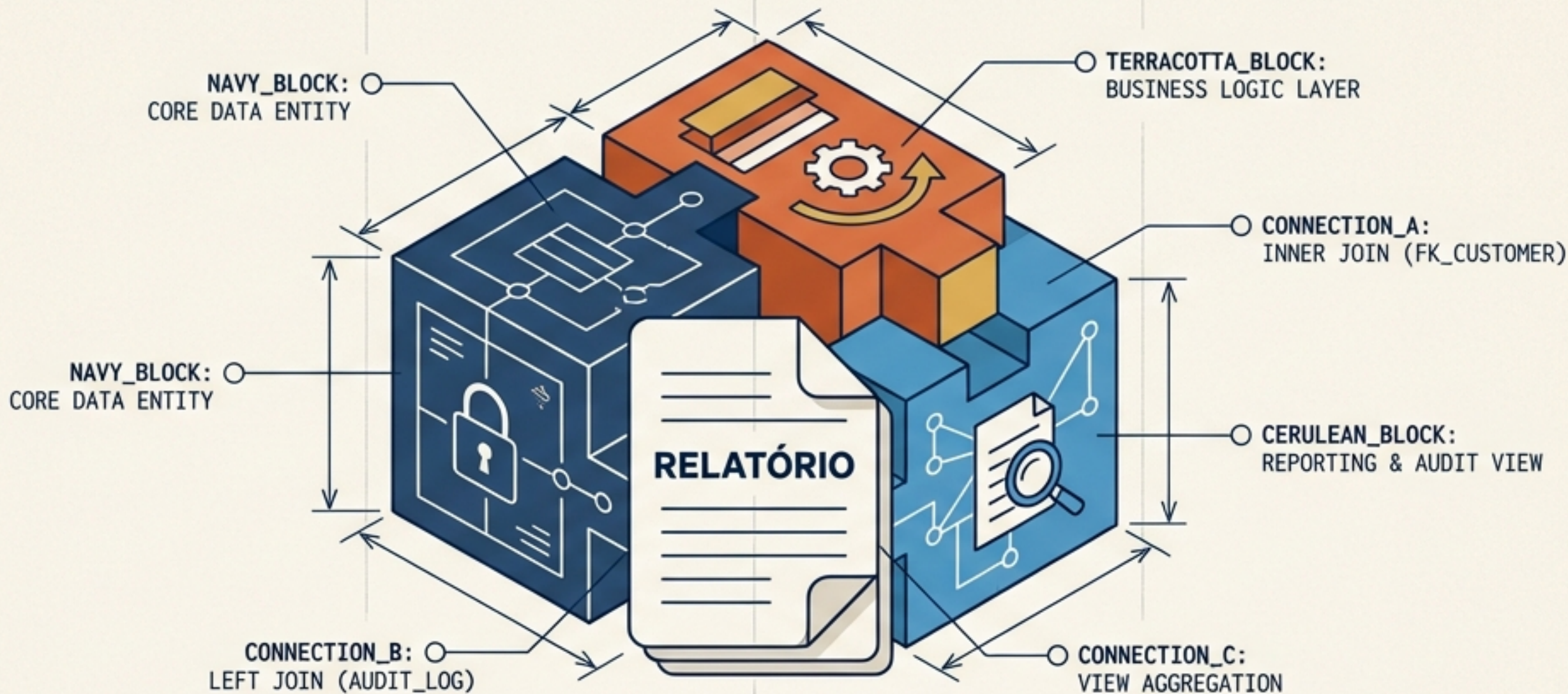




O BLUEPRINT DE DADOS: SQL JOINS NA LÓGICA DE NEGÓCIOS CORPORATIVA

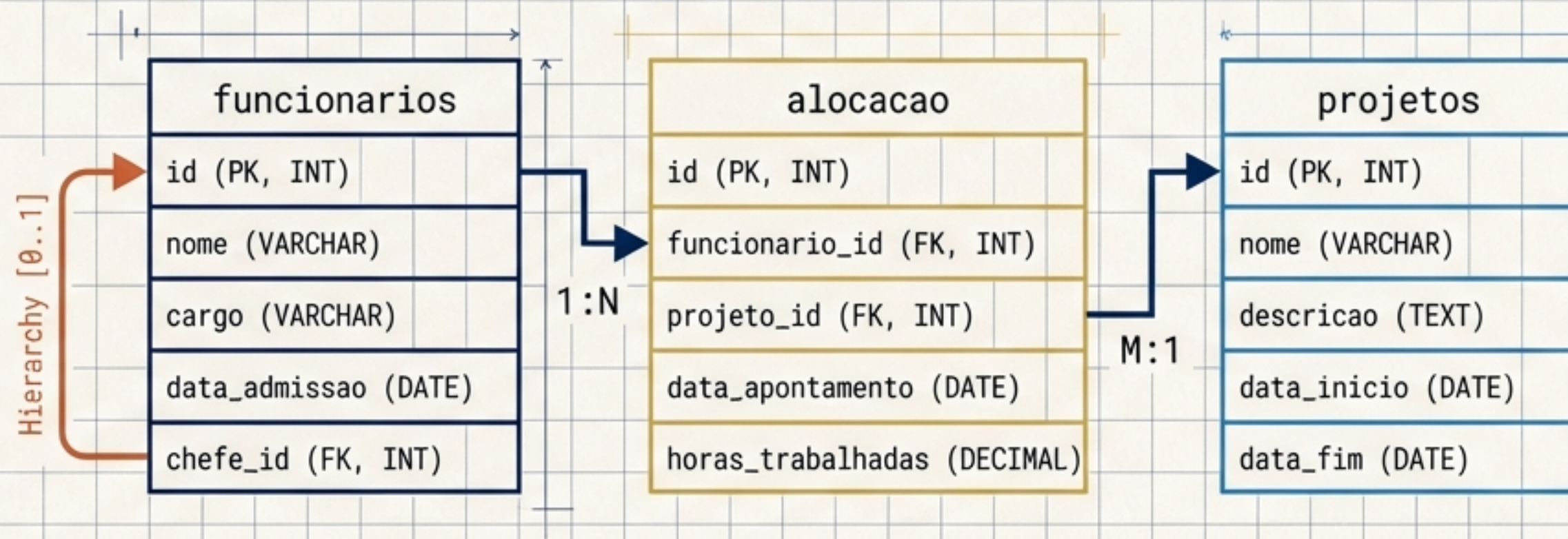
Um guia visual avançado do PostgreSQL para modelagem de relatórios, auditoria e APIs.

SCHEMA DEFINITION	SQL QUERY PATTERN	BUSINESS RULE	EXPECTED OUTPUT (JSON/TABLE)			

O Ecossistema Corporativo (Cenário Base)

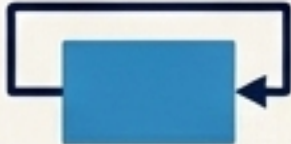
Nossas consultas irão operar sobre uma arquitetura corporativa clássica: uma hierarquia organizacional auto-referenciada (funcionário e chefe) acoplada a uma alocação muitos-para-muitos (apontamento de horas em projetos).

O domínio deste modelo resolve 90% dos desafios de relatórios e faturamento.



Matriz Diagnóstica de Operações Relacionais

A escolha do JOIN não é uma decisão técnica; é uma regra de negócio. Use esta matriz para traduzir imediatamente a pergunta gerencial para a sintaxe correta.

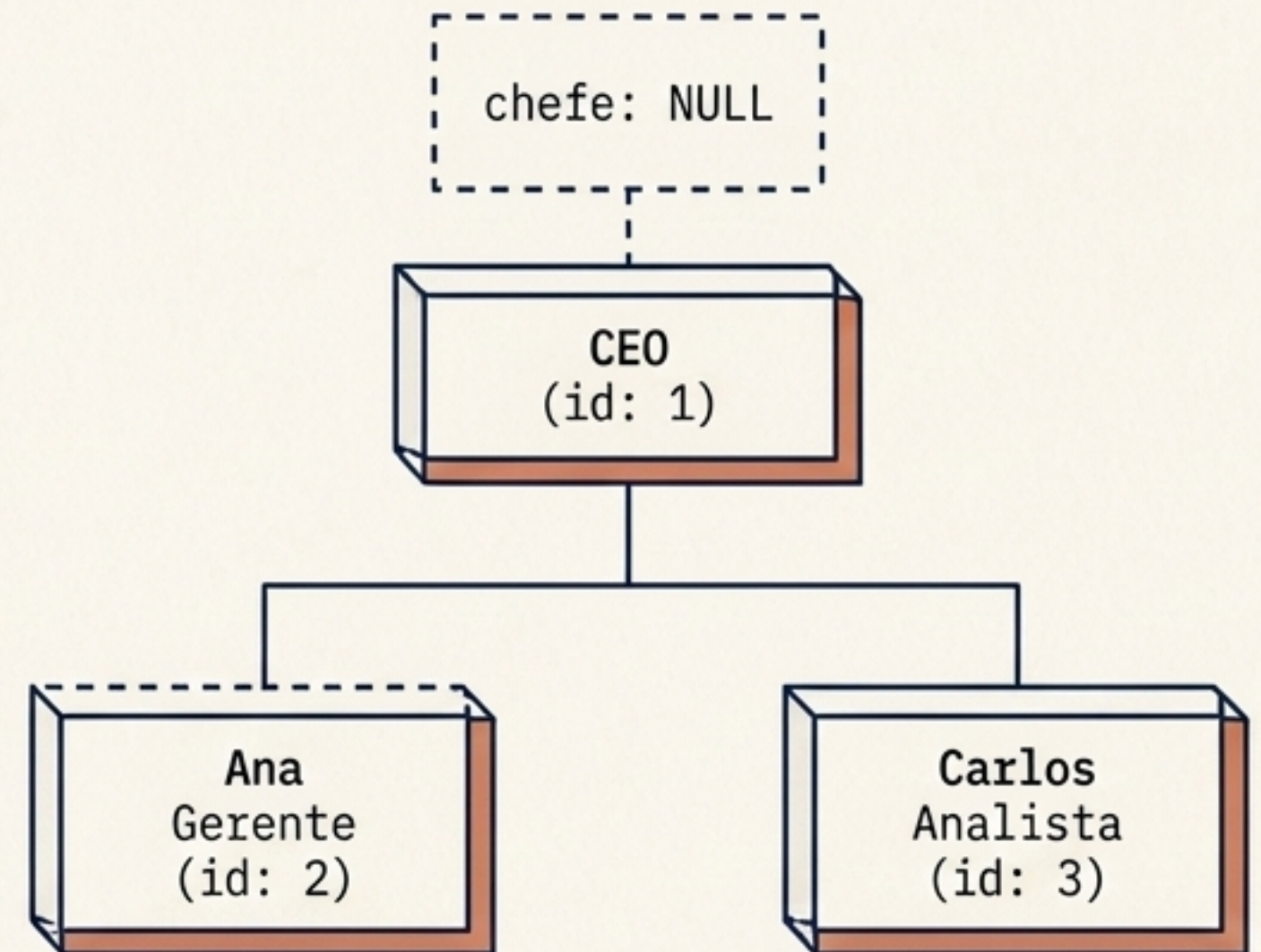
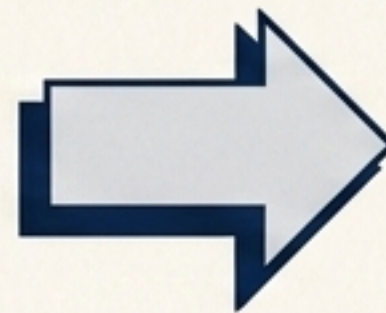
Tipo de JOIN	Mecanismo	Cenário de Negócio	Dados Retidos	Tratamento de Nulos
Inner		Produtividade	Apenas correspondências	Ignorados
Left		Folha de Pagamento	Todos da Esquerda	Adicionados à Direita
Right		Projetos Ociosos	Todos da Direita	Adicionados à Esquerda
Full Outer		Auditoria Geral	Todos os Dados	Adicionados em ambos
Self		Organograma	Correspondências na mesma tabela	Conforme a raiz
Anti		Funcionários Ociosos	Apenas Sem Correspondência	Apenas Nulos

Self Join: Construindo o Organograma Hierárquico

```
FROM funcionarios f1
LEFT JOIN funcionarios f2
ON f1.chefe_id = f2.id
```

Relaciona cada funcionário ao seu respectivo chefe. É o padrão fundamental para validação de estruturas organizacionais. A junção à esquerda (**Left Self Join**) garante que o CEO — que não possui chefe (NULL) — não seja excluído do relatório.

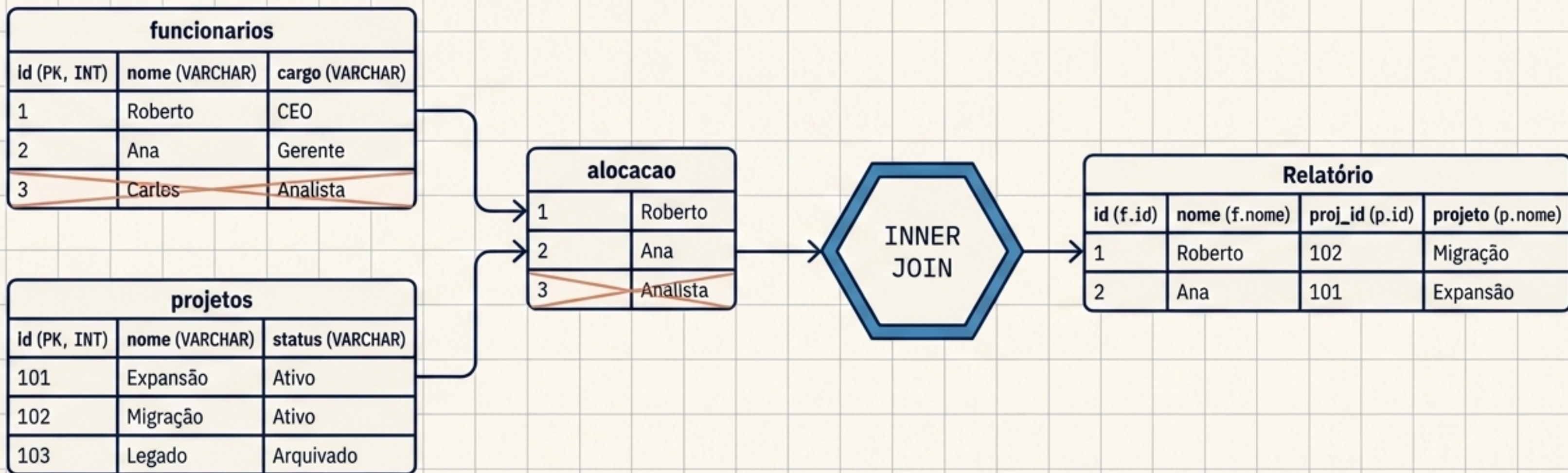
Tabela: funcionarios			
id (PK, INT)	nome (VARCHAR)	cargo (VARCHAR)	chefe_id (FK, INT)
1	Roberto CEO	CEO	NULL
2	Ana Gerente	Gerente	1
3	Carlos Analista	Analista	2



Inner Join: Alocação Ativa e Faturamento

```
SELECT * FROM funcionarios f
  INNER JOIN alocao a ON f.id = a.func_id
  INNER JOIN projetos p ON a.proj_id = p.id;
```

Retorna apenas os funcionários que possuem horas ativamente alocadas em projetos. Fundamental para a extração de relatórios de produtividade, controle de faturamento e cálculo de custo por projeto ativo. Nenhuma ociosidade passa pelo filtro.



Left Join: A Base Intocável da Folha de Pagamento

```
FROM funcionarios f  
LEFT JOIN alocacao a ON f.id = a.func_id  
LEFT JOIN projetos p ON a.proj_id = p.id;
```

O Left Join garante que a tabela de origem (esquerda) permaneça íntegra. Todos os funcionários aparecem no relatório, mesmo aqueles sem nenhuma alocação de horas no período. Em cenários de RH e folha de pagamento corporativa, a perda de um registro por falta de alocação seria um erro crítico.

Tabela: funcionarios		
id (PK, INT)	nome (VARCHAR)	cargo (VARCHAR)
1	Roberto	CEO
2	Ana	Gerente
3	Carlos	Analista

proj_id	projeto (p.nome)
102	Migração

proj_id	projeto (p.nome)
101	Expansão

NULL	
------	--

LEFT
JOIN

Relatório Final				
id (f.id)	nome (f.nome)	cargo (f.cargo)	proj_id (a.proj_id)	projeto (p.nome)
1	Roberto	CEO	102	Migração
2	Ana	Gerente	101	Expansão
3	Carlos	Analista	NULL	NULL

Funcionários com e sem alocações são preservados. O **NULL** indica ausência de dados na tabela da direita, mantendo o registro da esquerda ativo.

Right Join: Identificando Projetos Ociosos

A consulta inverte a âncora para garantir que todos os projetos sejam exibidos, independentemente de terem equipe alocada. É a ferramenta padrão para auditoria de portfólio: localizar imediatamente projetos financiados que estão paralisados ou sem cobertura de equipe técnica.

```
FROM projetos p
RIGHT JOIN alocao a ON a.proj_id = p.id
RIGHT JOIN funcionarios f ON f.id = a.func_id;
```

Tabela: projetos		
id (PK, INT)	nome (VARCHAR)	status (VARCHAR)
101	Expansão	Ativo
102	Migração	Ativo
103	Legado	Arquivado

func_id: 2
nome: Ana
cargo: Gerente

NULL

NULL

Alerta: Projeto Sem Equipe

RIGHT
JOIN

Relatório Final				
proj_id (p.id)	projeto (p.nome)	id (f.id)	nome (f.nome)	cargo (f.cargo)
101	Expansão	2	Ana	Gerente
102	Migração	NULL	NULL	NULL
103	Legado	NULL	NULL	NULL

Projetos com e sem alocações são preservados. O **NULL** indica ausência de dados na tabela da esquerda (funcionarios), destacando projetos sem equipe técnica.

Full Outer Join: Visão Panorâmica de Auditoria

Retorna todos os registros possíveis de ambas as tabelas. Na engenharia de dados, não é usado para relatórios operacionais comuns, mas sim como uma varredura de integridade. Revela simultaneamente funcionários sem trabalho e trabalhos sem funcionários em uma única execução.

```
FROM funcionarios f
FULL OUTER JOIN alocao a
  ON f.id = a.func_id
FULL OUTER JOIN projetos p
  ON a.proj_id = p.id;
```

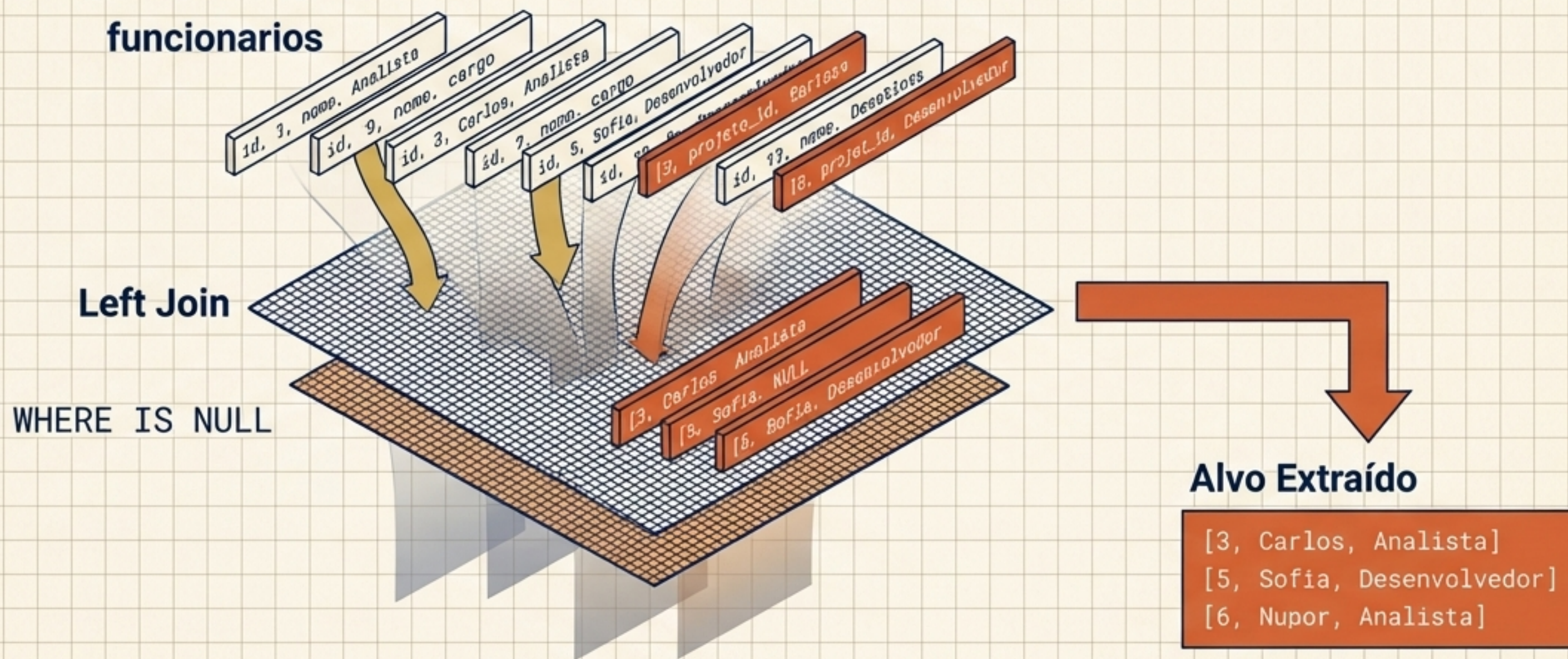
Relatório Final: Auditoria de Integridade				
Funcionários			Projetos	
id (PK, INT)	nome (VARCHAR)	cargo (VARCHAR)	proj_id (PK, INT)	projeto (VARCHAR)
1	Roberto	CEO	NULL	
3	Carlos	Analista	NULL	
2	Ana	Gerente	101	Expansão
5	Sofia	Desenvolvedor	102	Migração
Projeto Ocioso	NULL		103	Legado
	NULL		104	Pesquisa
	NULL			

Esta visão completa consolida todas as possibilidades: correspondências, funcionários sem projetos (esquerda), e projetos sem funcionários (direita). O NULL indica a ausência de dados em um dos lados, fornecendo um panorama imediato para auditoria de integridade.

Anti Join: O Filtro de Ociosidade e Anomalias

```
WHERE projeto_id IS NULL
```

Permite isolar cirurgicamente a ausência de dados. Responde de forma direta à pergunta gerencial:
"Quem são os funcionários na folha de pagamento que não participaram de nenhum projeto este mês?"



NOT EXISTS: A Arquitetura de Alta Performance

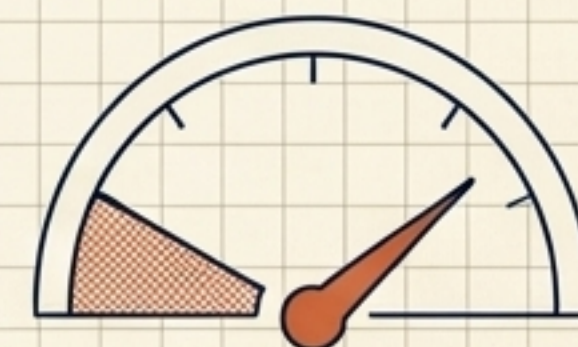
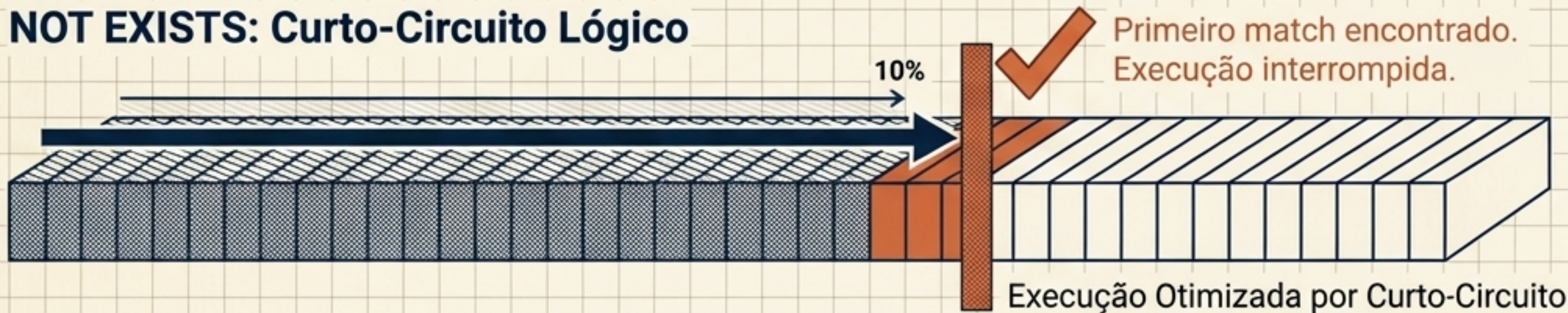
```
WHERE NOT EXISTS (SELECT 1 FROM alocacao... )
```

Em bancos de dados corporativos com milhões de registros de apontamento de horas, o Anti Join tradicional pode causar gargalos de memória. A abordagem NOT EXISTS atua como um circuito lógico: a varredura é interrompida assim que a **primeira alocação** é encontrada, otimizando drasticamente o tempo de resposta.

Anti-Join: Processamento em Lote



NOT EXISTS: Curto-Circuito Lógico



O Diagnóstico da Máquina: EXPLAIN ANALYZE

A criação de índices nas chaves de junção (**id**, **chefe_id**, **proj_id**) é o que transforma queries lentas em execuções instantâneas. O comando **EXPLAIN ANALYZE** não apenas estima, mas executa e audita a consulta, permitindo que engenheiros de dados identifiquem exatamente onde o banco de dados está sofrendo gargalos de leitura.

```
Query Text: EXPLAIN ANALYZE SELECT f.nome FROM funcionarios f LEFT JOIN alocacao a ON f.id = a.funcionario_id  
WHERE a.projeto_id IS NULL;
```

QUERY PLAN

Hash Anti Join (cost=215.00..870.00 rows=3333 width=16) (actual time=2.100..8.500 rows=2 loops=1)

Hash Cond: (f.id = a.funcionario_id)

-> Seq Scan on funcionarios f (cost=0.00..510.00 rows=10000 width=20) (actual time=0.020..1.200 rows=10000 loops=1)

-> Hash (cost=180.00..180.00 rows=3333 width=4) (actual time=2.050..2.050 rows=5 loops=1)

Buckets: 1024 Batches: 1 Memory Usage: 4kB

-> Seq Scan on alocacao a (cost=0.00..180.00 rows=3333 width=4) (actual time=0.015..1.800 rows=5 loops=1)

Filter: (projeto_id IS NULL)

Rows Removed by Filter: 9995

-> Index Scan using pk_proj on projetos (cost=0.15..8.20 rows=1 width=4) (actual time=0.010..0.012 rows=1 loops=1)

Gargalo Crítico: Requer Índice

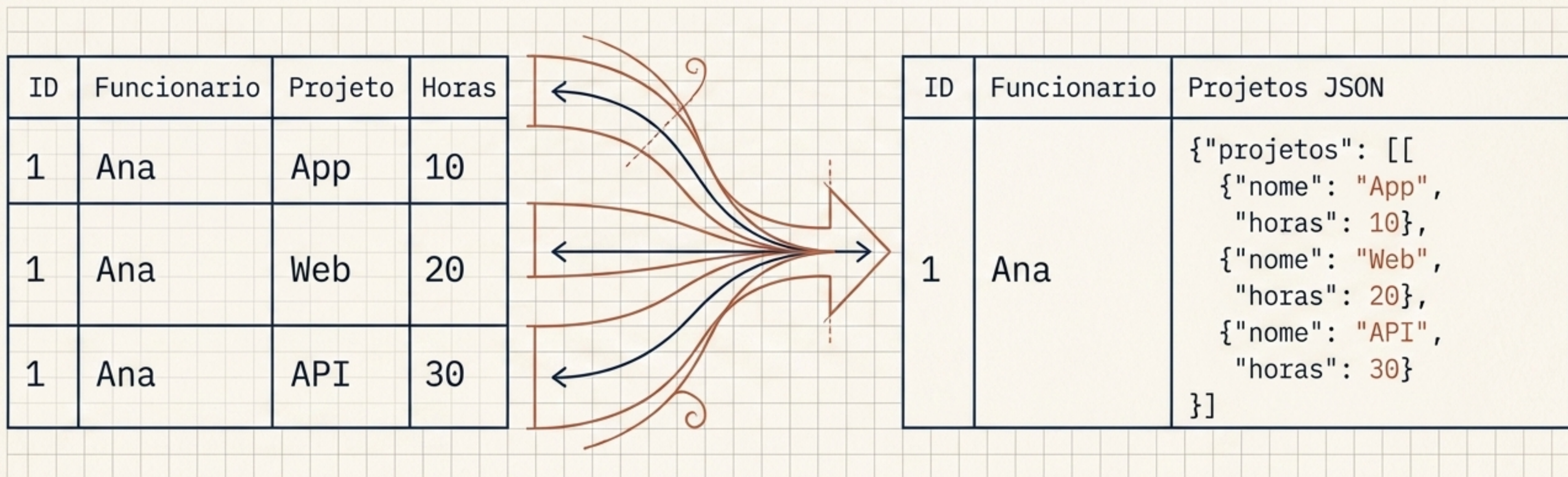
Leitura Otimizada

Gargalo Crítico: Requer Índice

Consolidação para APIs REST: Agregação em JSON

```
json_agg(json_build_object(...))
```

Dashboards e aplicações modernas web não consomem dados brutos repetidos. Ao consolidar os projetos de um funcionário em uma única linha estruturada com um array JSON, otimizamos o tráfego de rede e entregamos os dados no formato exato esperado pelo front-end.



A Prática Integrada: O Dashboard Executivo

A modelagem relacional transcende a sintaxe. Dominar estas junções significa ter o poder de projetar desde relatórios de faturamento precisos até integrações de arquitetura de microserviços. Cada variação do JOIN responde a um imperativo de negócio específico.

